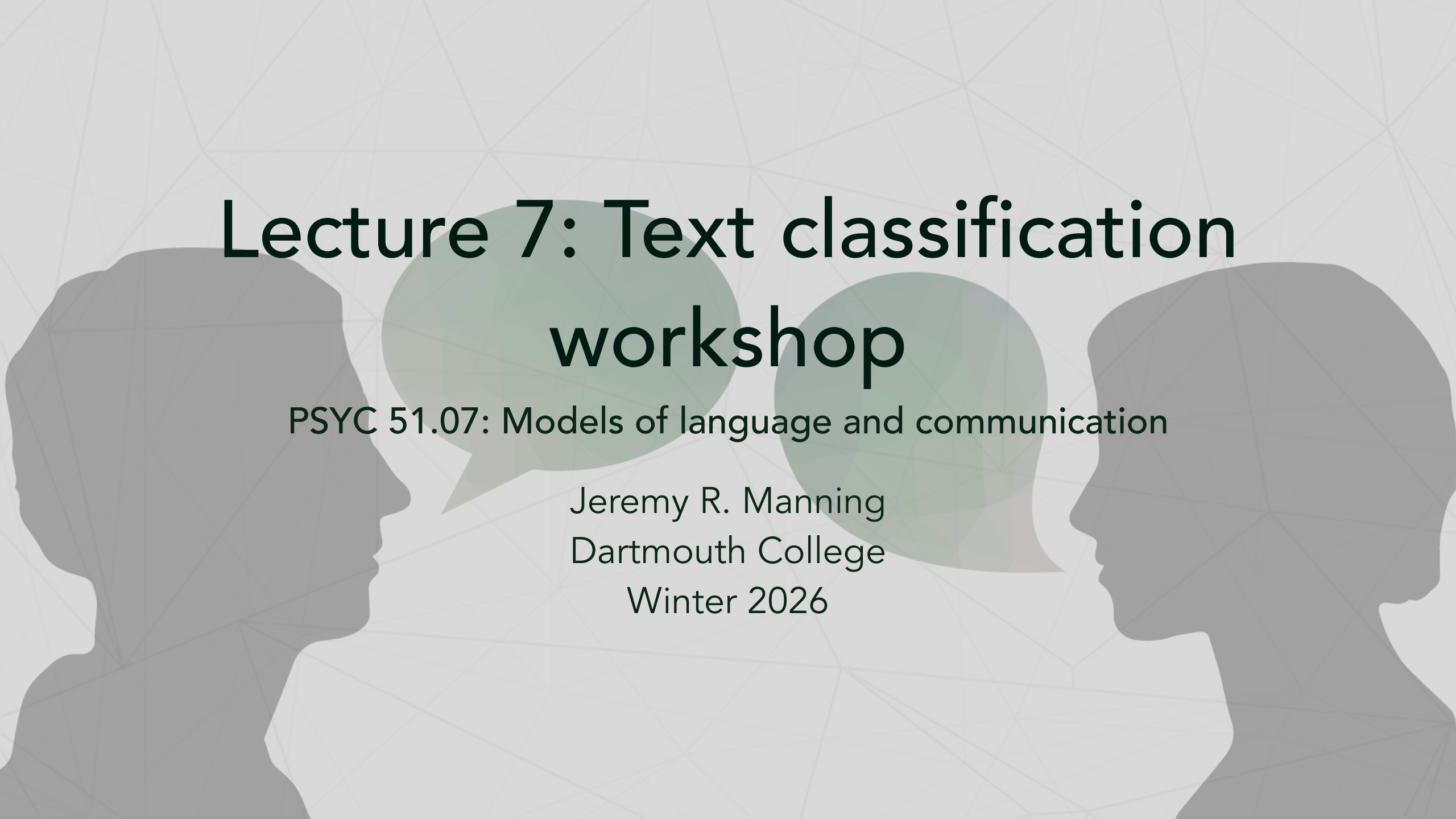


Lecture 7: Text classification workshop

PSYC 51.07: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning Objectives

By the end of this session, you will

1. Build text classifiers from scratch using scikit-learn
2. Understand different text representation methods (BoW, TF-IDF, embeddings)
3. Compare Naive Bayes, Logistic Regression, and Neural Network classifiers
4. Evaluate classifier performance using appropriate metrics
5. Debug common issues in text classification pipelines

Workshop format

Hands-on coding with the 20 Newsgroups dataset

Workshop Overview

Today's agenda

1. **Part 1:** Loading and exploring real data
2. **Part 2:** Feature engineering for text (BoW, TF-IDF)
3. **Part 3:** Building classifiers (Naive Bayes, Logistic Regression, Neural Networks)
4. **Part 4:** Model comparison and analysis
5. **Part 5:** Error analysis and improvements

Companion notebook

`xhour_classification_demo.ipynb` (click to open in Colab!)

Let's get started!

Let's dive in using the *notebook* instead of slides. After class, you can review these slides for reference.

The `20 Newsgroups` Dataset

A classic text classification benchmark

- Posts from 20 different newsgroups
- ~20,000 documents total
- Good for learning classification fundamentals

Today's subset (4 categories)

- `sci.space` — Science discussions about space
- `rec.sport.hockey` — Sports discussions about hockey
- `misc.forsale` — Random items for sale
- `comp.graphics` — Computer graphics

Why these?

Relatively distinct topics for easier learning.

Loading the Data

```python
from sklearn.datasets import fetch_20newsgroups

categories = [
    'sci.space',
    'rec.sport.hockey',
    'misc.forsale',
    'comp.graphics'
]

train_data = fetch_20newsgroups(
    subset='train',
    categories=categories,
    shuffle=True,
    random_state=42,
    remove=('headers', 'footers', 'quotes')  # Remove metadata
)

print(f"Loaded {len(train_data.data)} training documents")
```

Always explore your data before building models

Key questions to ask

1. How many documents per category?
2. What do the documents look like?
3. What words/phrases might be good indicators?
4. Are there categories that might be hard to distinguish?

Exploring the data: document counts and samples

```python
# Check class distribution
print("Documents per category:")
for i, name in enumerate(train_data.target_names):
    count = (train_data.target == i).sum()
    print(f"  {name}: {count}")
# sci.space: 593, misc.forsale: 585, comp.graphics: 584, rec.sport.hockey: 600

# Look at a sample document
print(train_data.data[1].strip())
# Didn't one of the early jet fighters have these?...
```

Think about it...

In our example dataset, the classes are roughly balanced (~600 documents each). Why is this good for training classifiers? What challenges might arise if one class had 10,000 documents and another only 100?

Convert text to numbers for machine learning

Two approaches today

1. **Bag of Words (BoW):** Count word frequencies
2. **TF-IDF:** Weight by document frequency

Think about it...

Remember yesterday's lecture on tokenization? How might the choice of tokens (words, n-grams, subwords, characters) affect these representations? For today, we'll use the word "word" to mean "token". But keep in mind that other tokenization strategies could be used!

Bag of Words: count how many times each word appears in each document

```python
from sklearn.feature_extraction.text import CountVectorizer

bow_vectorizer = CountVectorizer(
    max_features=5000,    # Keep only top 5000 words
    min_df=2,             # Word must appear in at least 2 docs
    max_df=0.8,           # Word must appear in <80% of docs
    stop_words='english' # Remove common words
)

X_train_bow = bow_vectorizer.fit_transform(train_data.data)
# Result: Sparse matrix of word counts
```

Bag of Words: observations, strengths, and weaknesses

- Simple and effective for many tasks
- Captures word presence and frequency
- Ignores word order and context
- Not all words are equally informative
- Usually high-dimensional and sparse

Term Frequency-Inverse Document Frequency

TF-IDF definition

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

where:

- t = term (word or token)
- d = document
- N = total number of documents
- $\text{df}(t)$ = number of documents containing term t
- $\text{TF}(t, d)$ = frequency of term t in document d
- $\text{IDF}(t) = \log \frac{N}{\text{df}(t)}$ = (log) inverse document frequency

The intuition

Downweight common words, upweight rare and informative words!

TF-IDF in practice

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(
    max_features=5000,
    min_df=2,
    max_df=0.8,
    stop_words='english',
    use_idf=True,
    sublinear_tf=True  # Use log scaling for term frequency
)

X_train_tfidf = tfidf_vectorizer.fit_transform(train_data.data)
# Result: Sparse matrix of TF-IDF scores
```

Three classifier approaches to compare

Today's candidates

1. **Naive Bayes:** Fast, probabilistic, good baseline
2. **Logistic Regression:** Linear, *interpretable*, performs well
3. **Neural Network:** Flexible, can learn complex patterns, can also overfit

Naive Bayes classifier

Bayes' theorem

$$P(y|x) = \frac{P(y)P(x|y)}{P(x)}$$

Bayes' theorem with independence assumption

$$P(y|x) \propto P(y) \prod_{i=1}^{n} P(x_i|y)$$

- **Why "naive"?** Assumes features are independent (they're not!)
- **Why does it work?** Despite the wrong assumption, it often performs well for text.

```
1 from sklearn.naive_bayes import MultinomialNB
2
3 nb = MultinomialNB()
4 nb.fit(X_train_tfidf, train_data.target)
```

Logistic regression classifier

Learns weights for each feature

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_j e^{w_j^T x}}$$

where

- w_k = weight vector for class k
- x = input feature vector (e.g., TF-IDF scores)

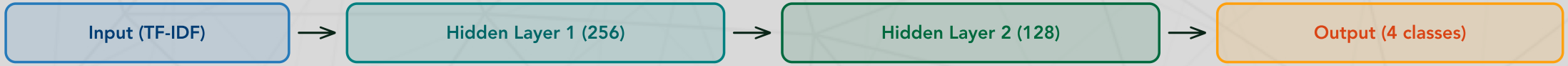
Advantages

- Interpretable weights: which words matter?
- Fast training and prediction

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train_tfidf, train_data.target)
```

Neural network classifier



A (simple) feedforward architecture

```
import torch.nn as nn

class TextClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim // 2)
        self.fc3 = nn.Linear(hidden_dim // 2, output_dim)
        self.dropout = nn.Dropout(0.3)
        self.relu = nn.ReLU()

nn_model = TextClassifier(input_dim=X_train_tfidf.shape[1], hidden_dim=256,
output_dim=4)
```

Evaluating classifier performance

- **Accuracy:** overall correctness: $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** of predicted positives, how many are truly positive?
- **Recall:** of actual positives, how many did we catch?
- **F1-Score:** harmonic mean: $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$ (Intuition: balances precision and recall)
- **Confusion Matrix:** detailed breakdown of predicted versus actual labels

Confusion matrix

Visual representation of classifier errors

	Predicted A	Predicted B	Predicted C	Predicted D
Actual A	85	2	3	0
Actual B	1	92	2	5
Actual C	4	3	88	5
Actual D	0	8	2	90

Interpretation

Diagonal = correct predictions. Off-diagonal = errors.

Error analysis: critical but often skipped

1. Find misclassified examples
2. Look for patterns
3. Understand why the model failed
4. Use insights to improve

Key Takeaways

1. **Good features matter more than complex models** (for many tasks)
2. **TF-IDF usually beats raw BoW** for text classification
3. **Linear models are competitive** with neural networks on bag-of-words
4. **Always examine errors** to understand model behavior

Putting this week's lectures into context

This week's pipeline so far

1. Monday: Data cleaning (Lecture 5)
2. Wednesday: Tokenization (Lecture 6)
3. **Today:** Classification (Lecture 7)
4. Friday: POS Tagging & Sentiment Analysis (Lecture 8)

Up next...

Part-of-speech tagging and sentiment analysis: how do these building blocks combine for real NLP applications?

Hands-on exercise

Open the companion notebook

`xhour_classification_demo.ipynb`

Steps

1. Load the 20 Newsgroups dataset
2. Experiment with different vectorizers
3. Train multiple classifiers
4. Analyze errors and improve
5. Try your own text examples!

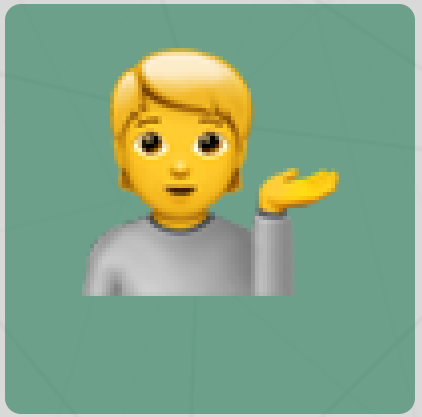
Goal

Build intuition for text classification

Questions? Want to chat more?

Email me

Join our Discord



Come to office hours

Reminder

ELIZA assignment is due *tomorrow* at 11:59pm!